CHAPTER 12 | SOFTWARE ENGINEERING, MIS & E-COMMERCE

12.1 Software Engineering and the Development Life Cycle

Software Engineering: the systematic, disciplined approach to developing software. Its primary goal is to build software that **meets user requirements within budget and schedule**, and remains correct, reliable and maintainable afterwards. Writing code fast is only a small part of it.

SDLC: System Development Life Cycle — the sequence of phases every software system passes through, from the first discussion to retirement. The order of phases is fixed:

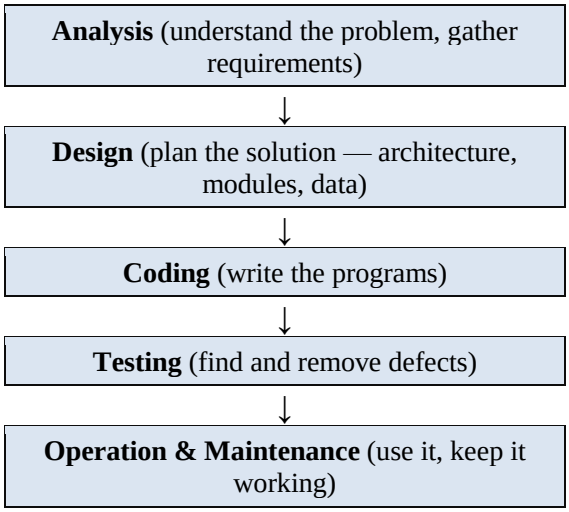


Fig. 12.1 — The five phases of SDLC. Analysis always comes before Design; Testing always comes before Maintenance.

Feasibility Study: done before the project starts. It answers one question — should we build this at all? It checks four things: **technology, finance (economic), time and resources**. Project size is judged under **technical feasibility**; cost-benefit is **economic feasibility**.

Software Configuration Management (SCM): the activity that **identifies, controls, audits and reports every modification** made to the software, during development and after release. It keeps all versions consistent.

Reverse Engineering: studying the **existing code to recover its design and requirements** — used to understand and document old (legacy) software whose documents are lost.

Re-Engineering: one step further — the old system is understood and then **rebuilt in a new form**, without changing what it does.

Remember: Reverse engineering only *understands* the system; re-engineering *rebuilds* it.

Exam Points — 12.1

- ✦ Primary goal of software engineering = software that meets user requirements within budget and schedule.
- ✦ SDLC = System Development Life Cycle; correct order: Analysis → Design → Coding → Testing → Operation & Maintenance.
- ✦ SCM identifies, controls, audits and reports modifications. Feasibility study checks financial + technical viability (project size → technical feasibility).
- ✦ Recovering design from code = software reverse engineering; reconstituting a system in a new form = re-engineering.

12.2 SDLC Models

A process model tells us **in what order and in what manner** the SDLC phases will be performed. Different projects need different models.

(A) Waterfall Model (Linear Sequential Model)

The **oldest** software engineering paradigm, given by **Royce**. Work flows strictly downward — each phase finishes completely before the next begins, like water falling down steps. There is no going back.

(1) Requirements must be **frozen before design begins** — that is its basic assumption.
(2) So it is **incompatible with changing requirements** — any linear model is.
(3) Nature: **long duration, high risk, but low cost** for well-understood projects.

(B) Prototype Model

When the customer is not sure what he wants, we first build a quick working model — a **prototype** — show it, take feedback, refine it, and only then engineer the real product.

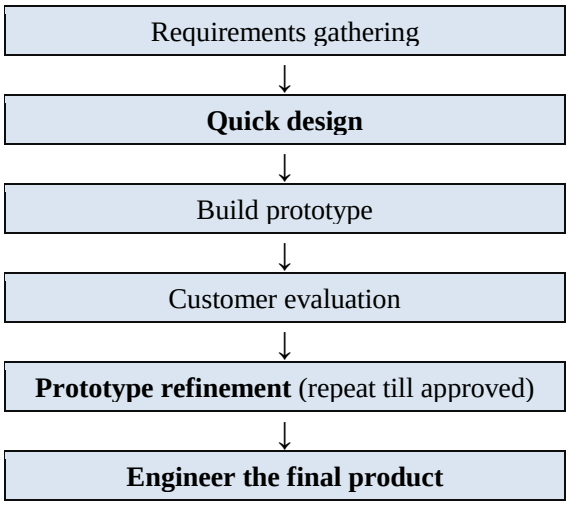


Fig. 12.2 — Prototype model. Note: full 'Coding' is NOT a phase of prototyping — the real product is engineered only at the end.

(1) A prototype serves **all three purposes together**: demonstrates functionality to stakeholders, gives users a working model to test, and checks technical feasibility.
(2) **Best model when requirements keep changing** — the user reacts to something concrete.
(3) Two flavours of evolutionary development: **exploratory development** and **throwaway prototyping**.

(C) Incremental Model

The product is delivered in pieces (increments). Increment 1 is the core product; every later increment adds functionality. It is a **combination of the Linear (waterfall) model and the Prototyping model** — each increment is built linearly, and the series of increments behaves like prototyping.

(D) RAD Model (Rapid Application Development)

(1) A **"high-speed" adaptation of the linear sequential model** — it emerged in response to the weaknesses of the waterfall model.
(2) Its phases: **Business modelling → Data modelling → Process modelling → Application generation → Testing**. ("Development modelling" is not a RAD phase.)
(3) RAD is **not suitable when technical risks are high**, because it depends on assembling well-understood, reusable components quickly.

(4) RAD and Prototyping both **fail without active user participation**.

(E) Spiral Model

Given by **Barry Boehm (1986)**. Development moves in loops of a spiral; every loop has four activities — planning, **risk analysis**, engineering, customer evaluation. Software is released in a **series of incremental releases**, one per loop.

(1) Its most important feature is **risk management** — it is the only model that gives **explicit recognition of risk** on every cycle.

(2) It is also called the **meta model**, because other models (waterfall, prototyping) can fit inside its loops.

(3) It **couples the iterative nature of prototyping with the controlled, systematic aspects of the linear sequential model** — Boehm's own description.

(F) Agile, 4GT and Other Models

(1) **Agile** = a methodology that emphasises **flexibility and adaptability to change** — short iterations, customer collaboration, working software over documentation. Scrum and XP (Extreme Programming) are its popular frameworks.

(2) **4GT (Fourth Generation Techniques)** model = a set of **software tools** that generate code from a high-level specification.

(3) **V-Model** (companion of waterfall): every development phase on the left arm has a matching testing phase on the right arm — requirements↔acceptance testing, design↔integration testing, coding↔unit testing.

(4) Valid life-cycle models: Waterfall, Iterative, Prototype, Incremental, RAD, Spiral, Agile, JAD, V-Model. **"Circular" is not a model.**

(5) Pressman's five generic framework activities, in order: **Communication → Planning → Modelling → Construction → Deployment**.

Model	Best when	One-line identity
Waterfall	Requirements stable and well understood	Linear; requirements frozen first; Royce
Prototype	Requirements unclear / changing	Build → show → refine → then engineer
Incremental	Product deliverable in useful pieces	Linear + Prototyping combined
RAD	Fast delivery, low technical risk	High-speed waterfall; heavy user involvement
Spiral	Large, risky projects	Iterative + risk analysis; Boehm; meta model
Agile	Change is constant	Flexibility and adaptability

MVC: Model–View–Controller — a design architecture. Model keeps the data, View displays it, Controller handles user input and connects the two.

Behaviour Model: in system modelling, the model that shows **how the software responds to external events** (state diagrams). The context model shows system boundaries; the data model shows information.

Exam Points — 12.2

- ✦ Waterfall = linear sequential = Royce; assumes frozen requirements; low cost but long duration and high risk.
- ✦ Spiral = Boehm = meta model = risk model; incremental releases; explicit risk analysis every loop.
- ✦ RAD phases: business, data, process modelling + application generation + testing; unsuitable when technical risk is high.
- ✦ Incremental = Linear + Prototyping. Prototype phases: quick design, build, refinement, engineer product (no separate 'coding' phase).
- ✦ Generic process: Communication, Planning, Modelling, Construction, Deployment. 4GT = software tools. MVC = Model View Controller.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

12.3 Requirements Engineering, DFD and UML

Before we design anything, we must know exactly **what** the customer wants. Requirements engineering has four activities, done in this order: **Feasibility study → Requirements elicitation & analysis → Requirements specification → Requirements validation**. (Component specification is a design activity, not a requirements activity.)

SRS: Software Requirements Specification — the agreed document stating what the system must do. It is a **black-box specification**: it describes external behaviour only, never the internal working. It contains functional requirements, non-functional requirements and goals of implementation — but **never algorithms** (algorithms belong to design).

Types of requirements: there are three recognised categories:

(1) Functional requirements — what the system must do (its features).

(2) Non-functional requirements — how well it must do it: maintainability, portability, robustness are all non-functional. Their sub-types: **product, organisational and external** requirements ("umbrella" is not a type).

(3) Domain requirements — coming from the field of application. ("Physical requirement" is not a category.)

Elicitation techniques: interviews, questionnaires, observation, prototyping and **brainstorming** — the group technique where stakeholders throw ideas freely.

Data Flow Diagram (DFD)

A DFD shows how data moves through the system. Because its processes are drawn as circles (bubbles), the DFD is also called a **bubble chart**. Its four symbols:

Symbol	Meaning
Circle / bubble	**Process** — transforms the data
Arrow	**Data flow** — data in motion
Two parallel lines (open rectangle)	**Data store** — data at rest; it is simply a **repository of data**, whatever the medium (file, table, memory)
Rectangle	**External entity** — source or sink outside the system

Note: The **Level-0 DFD is called the context diagram** — the whole system drawn as a single bubble with its external entities. Level-1 breaks that bubble into major processes, Level-2 breaks those further.

UML and CASE Tools

UML: Unified Modeling Language — the standard notation (not a process) for object-oriented design. In a **sequence diagram**, the participating objects are drawn as boxes across the top, each with a vertical dashed lifeline, and messages flow between them downward in time order.

CASE tools: Computer Aided Software Engineering tools — software that automates development work. They **improve quality, reduce the drudgery of the engineer's work, and reduce maintenance cost**.

Requirements model elements: scenario-based, class-based, behavioural and flow-oriented elements. ("Usage-based" is not one.)

Remember: The **linker** resolves externally defined symbols between separately compiled modules (compile → assemble → link → load).

Exam Points — 12.3

- ✦ SRS = black-box specification; includes functional + non-functional requirements + implementation goals; excludes algorithms. SRS full form: Software/System Requirements Specification.
- ✦ DFD = bubble chart; data store = repository of data; Level-0 DFD = context diagram.
- ✦ Maintainability, portability, robustness → non-functional. Non-functional sub-types: product, organisational, external.
- ✦ Requirement engineering phases: feasibility, elicitation & analysis, specification, validation — not component specification. Brainstorming = group elicitation technique.
- ✦ UML = Unified Modeling Language; sequence diagram = objects on top with dashed lifelines. CASE = Computer Aided Software Engineering.

12.4 Software Design — Cohesion, Coupling and Architecture

Cohesion: the measure of **functional strength of a module** — how strongly the elements inside one module belong together. **High cohesion is good.**

Coupling: the measure of **degree of functional interdependence (interaction) between two modules**. **Low coupling is good.**

Golden rule of design: keep **coupling LOW and cohesion HIGH**. Loosely coupled, highly cohesive modules are the easiest to understand and maintain.

Teachers remember the two ladders — from best to worst:

Cohesion (best → worst)	Meaning
Functional	All elements achieve one single function — the ideal
Sequential	Output of one element is input of the next
Communicational	Elements work on the same data
Procedural	Elements are parts of one procedure
Temporal	Elements run at the same time (e.g. initialisation)
Logical	**Elements perform similar category of operations** (all error handling, all data input etc.)
Coincidental	Elements have no relation — the worst

Coupling (best → worst)	Meaning
Data	Modules pass only elementary data items — the best
Stamp	Modules pass a composite data structure
Control	One module passes a flag controlling the other
Common	**Modules share the same global data area**
Content	One module uses the code/data inside another — the worst

Design pattern: a **standard, reusable solution to a common design problem** — proven once, applied everywhere.

Structured partitioning: a program should be partitioned **both horizontally** (into major functional branches) **and vertically** (into a control hierarchy).

Pipe-and-filter architecture: each processing component (filter) is discrete, performs one transformation, and passes the data through a pipe to the next — the Unix pipeline is the classic example. Other styles: repository, layered, client-server.

Exam Points — 12.4

- ✦ Cohesion = functional strength of a module; Coupling = interdependence between modules; aim low coupling + high cohesion.
- ✦ Logical cohesion = elements perform similar operations (error handling, data input). Common coupling = shared global data area.
- ✦ Design pattern = standard solution to a common problem. Structured partitioning = horizontal + vertical. Pipe-and-filter = discrete transforms in a chain.
- ✦ Software evolution has both a software-architecture aspect and a programming-language aspect.

12.5 Software Testing

Verification: "Are we building the product **right**?" — checking the work against its **specification**.

Validation: "Are we building the **right** product?" — checking it against the **user's actual need**. (Boehm's definitions; asked repeatedly.)

The four levels of testing are always performed in this order:

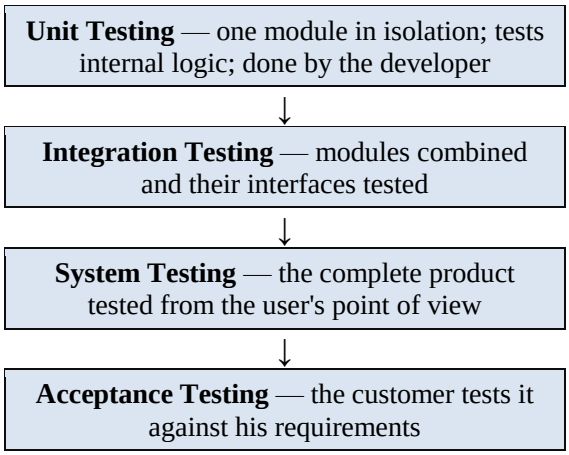


Fig. 12.3 — Levels of testing. Development testing = unit + component + system; release testing comes AFTER development testing and BEFORE acceptance testing.

Black-box testing	White-box testing
Tests from the **specification** only — internal code not seen	Tests the **internal logic and code structure**
Also called closed-box testing	Also called **glass-box** (clear-box) testing
Methods: **equivalence class partitioning** and **boundary value analysis** (the two main approaches); cause-effect graphing	Methods: basis path testing, loop testing, data-flow testing

Worked example (BVA): to test a program valid for the range 1 to 4000, boundary value analysis picks the edges and just-outside values: **0, 1, 4000, 4001**.

Alpha testing: done **at the developer's site**, in a controlled way, with the developer watching.

Beta testing: done **at the customer's (end-user's) site**, in the real environment, developer absent.

Regression testing: re-running the old tests after a change, to confirm the **change has not broken old functionality**.

Recovery testing: forces the software to fail in various ways and **verifies that recovery is performed properly**.

Mutation testing: a **fault-based** technique — small faults are deliberately planted in the code; if the tests do not catch them, the test suite is weak.

Test-Driven Development (TDD): the programmer **writes the test scripts first**, then writes code to pass them (red → green → refactor).

Code walk-through: an **informal** technique of analysing code, done **after** coding of the module (not before).

(1) Integration approaches: top-down, bottom-up, sandwich, functional incremental. **Big-bang** combines everything at once — it is **not** an incremental approach; in big-bang testing all modules are integrated and tested as a complete system in one go.

(2) Goal of testing: **maximise errors detected (A) while minimising test cases used (B)**.

(3) Testing levels are unit, integration, system, acceptance — there is no "proper testing".

SQA: Software Quality Assurance — establishing and enforcing **standards and processes** so that quality software is produced. QA prevents defects; quality control (QC) detects them in the product.

Exam Points — 12.5

- ✦ Verification = building the product right (spec); Validation = building the right product (need).
- ✦ White box = glass box; black box works from program specification; its two main methods = equivalence partitioning + boundary value analysis; BVA for 1–4000 → 0, 1, 4000, 4001.
- ✦ Unit tests internal logic of one module; regression protects old functionality; recovery testing forces failure and checks recovery; mutation testing is fault-based; TDD writes tests first.
- ✦ Alpha at developer's end; beta at user's site. Big-bang is not incremental. Release testing is a black-box test done before acceptance, by the developer.
- ✦ SQA = standards and processes for quality. Walk-through = informal, after coding.

12.6 Software Quality, Maintenance and CMM

Quality factors: the accepted set is functionality, reliability, usability, **efficiency** (optimal use of resources — sub-attributes: **time behaviour and resource behaviour**), maintainability and portability.

Process quality vs product quality: visibility, timeliness and rapidity describe the **process**; **portability describes the product**, so it is not a process quality.

ISO 9000: the framework of **guidelines for achieving consistent, high-quality software** — it certifies the *process*, not the product.

CMM: Capability Maturity Model — rates the maturity of an organisation's software process on five levels. Learn the ladder:

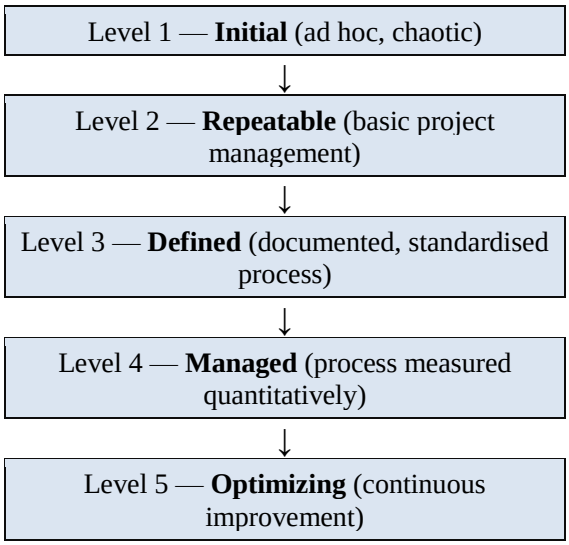


Fig. 12.4 — The five CMM maturity levels.

Software patch: an **emergency fix** — a small piece of code applied quickly to repair **bugs or security vulnerabilities**. Software updates in general are also called **patching**.

Maintenance categories: there are exactly four:

(1) Corrective — fixing faults found after delivery.

(2) Adaptive — adjusting the software to a new environment (new OS, new hardware).

(3) Perfective — improving performance or adding refinements users ask for.

(4) Preventive — restructuring today to prevent problems tomorrow. ("Effective maintenance" does not exist.)

Evolutionary development: builds the system through successive versions; its two flavours are **exploratory development and throwaway prototyping**. Suits small/medium projects with unclear requirements.

Exam Points — 12.6

✦ Efficiency = time behaviour + resource behaviour. Portability is a product quality, not a process quality.
✦ ISO 9000 = process guidelines for consistent quality. CMM = Capability Maturity Model.
✦ Patch = emergency fix for bugs/security holes; updates = patching.
✦ Maintenance: corrective, adaptive, perfective, preventive — 'effective' is not a category.
✦ Evolutionary development = exploratory + throwaway prototyping. CASE tools improve quality and cut maintenance cost.

12.7 Project Management, Estimation and Reliability

Scheduling — Gantt and PERT

(1) A **Gantt chart** lays tasks against a calendar; it is prepared in the **planning phase**.

(2) A **PERT chart** is a network of **boxes (activities) and arrows (task dependencies)**.

(3) A Gantt chart **can** be derived automatically from a PERT chart, but a PERT chart **cannot** be derived from a Gantt chart (the Gantt has no dependency information). So the common statement pair — 'PERT from Gantt' and 'Gantt cannot come from PERT' — is **both false**.

W5HH principle: Boehm's seven questions for a project plan — Why, What, When, Who, Where, How, How much. It covers the **objective, schedule, responsibilities**, approach and resources — all of them.

Risk management: runs as: risk **identification** (find them) → risk **analysis** (assess the **likelihood and consequences** of each) → risk **planning** (decide the response) → risk **monitoring**. The methodologies for handling a risk are **acceptance, avoidance and transfer** — "risk generation" is not one.

Staffing level estimation: decides how to **allocate appropriate resources (people) to the project** phase by phase.

Remember: Brooks's law — adding manpower to a late project makes it later.

Cost Estimation — COCOMO and Function Points

COCOMO: Constructive Cost Model, given by **Boehm**. It is an empirical (formula-based) model — not a decomposition method. Basic COCOMO estimates effort from size:

$$E = a \times (KLOC)^{b} \quad person-months$$

where KLOC = thousands of lines of code and the constants a, b depends on the project mode. The tested instance:

$$E = 3.2 \times (KLOC)^{1.05}$$

is keyed in exams as the **Boehm simple model**. The three COCOMO modes:

Mode	Project nature
Organic	**Small size, well-understood application, small experienced team** — the asked one
Semidetached	Medium size, mixed team

Embedded	Large, complex, tight hardware/regulation constraints

Estimation techniques: empirical, heuristic and analytical techniques are the recognised families; **prototyping is not an estimation technique**. **Decomposition methods** = LOC estimation and FP estimation; **COCOMO is not a decomposition method**.

Function Points (FP): measure software size from its **functionality** (inputs, outputs, files, enquiries, interfaces) — so size can be estimated **before any code exists**, and the measure is **language-independent** (LOC is not).

Software metrics: direct measures = concrete quantities: **lines of code, cost, effort, execution speed, memory size**. **Indirect measures** = derived qualities: **functionality, quality, efficiency, reliability**. (Exam statements often swap the two — then both are false.)

Effort: always measured in **person-months** — people multiplied by time, not months alone.

Reliability Measures

(1) MTTF = Mean Time To Failure — average operating time before a failure.
(2) MTTR = Mean Time To Repair — average time to fix it.
(3) MTBF = Mean Time Between Failures, and the three are connected:

$$MTBF = MTTF + MTTR$$

(4) ROCOF = Rate of Occurrence of Failure — how often failures appear.

FURPS: Hewlett-Packard's quality attributes — **F**unctionality, **U**sability, **R**eliability, **P**erformance, **S**upportability. **Portability is not in FURPS** (it enters only in FURPS+).

Fog Index: a metric of **document readability**, computed from average sentence length and word length.

Exam Points — 12.7

- Gantt → planning phase; PERT = boxes (activities) + arrows (dependencies); Gantt derivable from PERT, never the reverse.
- W5HH covers objective + schedule + responsibilities — all of these. Risk analysis = likelihood + consequences; methods = accept, avoid, transfer.
- COCOMO = Constructive Cost Model (Boehm); E = 3.2(KLOC)^1.05 = Boehm simple model; Organic mode = small familiar project, small experienced team.
- FP estimates size from functionality, language-independent. Direct measures = LOC/cost/effort; indirect = functionality/quality. Effort in person-months.
- MTTF = Mean Time To Failure; ROCOF = Rate of Occurrence of Failure. FURPS excludes portability. Fog index = readability. Prototyping is not an estimation technique.

12.8 MIS, DSS, Knowledge Management and Expert Systems

Information System: a set of interrelated components that **collect, process, store and distribute information** to support decision-making and control in an organisation. The **organisation** itself is the formal social unit devoted to attaining specified goals.

Attributes of good information: accuracy, timeliness, relevancy, completeness. (**"Isolation" is not an attribute.**) The **representational quality** of information covers **interpretability, format, coherence and compatibility**.

Levels of information: three levels, matching the levels of management:

(1) Strategic information — for **long-term policy decisions** that define objectives and check how well they are met (top management).
(2) Tactical information — medium-term control (middle management).

(3) Operational information — day-to-day working (lower management).

Types of Information Systems

System	Purpose
TPS (Transaction Processing System)	**Records, validates, processes and stores** the day-to-day transactions of the business for future retrieval — the base level
MIS (Management Information System)	A **planned system of collecting, storing and disseminating data** in the form of information needed to carry out the functions of management
DSS (Decision Support System)	Improves the company's **decision-making** capability; its components include the **user interface and the knowledge engine**
OAS (Office Automation System)	**Word processing, electronic filing, e-mail, message switching, data & voice communication** — office work, not annual budgeting
ESS/EIS (Executive Support System)	Summarised strategic information for top executives

(1) MIS **resources**: hardware, software, people and data — all four. Its **components** include an external database; "education procedures" is not a component.

(2) MIS **sources of information** are categorised as **internal and external**.

(3) Prerequisites of a good MIS: database, qualified MIS executives, control & maintenance, and support from **top management** — "support from staff" is the odd one out.

(4) TPS is a genuine **type** of information system; online/real-time/distributed describe processing modes, not IS types.

Knowledge Management (KM)

Knowledge: expertise and skills acquired **through experience or education** — theoretical or practical understanding of a subject.

(1) KM = the set of processes to **create, gather, store, transfer and apply** knowledge. Its key processes: knowledge **gathering, storage & organisation, distribution, and use** — all four.

(2) **Knowledge refining** = the step that adds value by **discovering relationships, abstracting and sharing**.

(3) **Tacit knowledge** is highly personal, gained by experience, **hard to formalise**; **explicit knowledge** is documented and easily shared; implicit sits between.

(4) KM's two-dimensional perspective covers **strategy and content**; its purposes are driving strategy, integration and sharing — **portability is not a purpose**.

(5) Building a KMS requires a **knowledge contract**; the **lowest-cost route is in-house development**. The management point of view = determining, organising, directing and monitoring knowledge practices.

(6) **Best practices** = an organisation's or industry's **most successful solutions or problem-solving methods**.

Expert Systems and Related Systems

(1) An expert system has three parts: **knowledge base** (what is known), **inference engine** (**applies logical rules to derive conclusions**), and **user interface**.

(2) Its capability: **suggesting alternative options** to a problem — it cannot refine its own knowledge or replace the human decision-maker. Its characteristics: understandable, highly responsive, high-performance and **reliable** ("unreliable" is the odd one).

(3) The **intelligent editor** is the interface between the **domain expert and the expert system** for entering knowledge.

System Analyst: the person who **analyses the business situation, identifies opportunities for improvement and designs the information system** — the analyst decides what and why; the programmer decides how.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

CRM: Customer Relationship Management — the **methodologies, technology and capabilities** that help an enterprise manage customer relationships. (ERP manages internal resources; SCM manages the supply chain.)

Information processing: the acquisition, storage, organisation, retrieval, display and dissemination of information.

Exam Points — 12.8

- ✦ TPS records/validates/stores transactions; MIS = planned system collecting, storing, disseminating information; DSS improves decision-making (user interface + knowledge engine); OAS = word processing, e-filing, e-mail.
- ✦ MIS resources: hardware, software, people, data. Sources: internal + external. Strategic information = long-term policy. Representational quality = interpretability, format, coherence, compatibility.
- ✦ KM processes: gather, store & organise, distribute, use; refining = discovering relationships + abstracting + sharing; tacit = personal, hard to formalise; best practices = most successful methods.
- ✦ Expert system: inference engine applies rules and derives conclusions; intelligent editor interfaces the domain expert; it suggests alternatives and must be reliable.
- ✦ System analyst analyses the business and designs the IS. CRM = methodologies + technology for customer relationships.

12.9 E-Commerce and E-Payment

E-Commerce: buying and selling goods and services over the internet. Its functions span **finance, supply chain and marketing** together. Its primary process, step by step: enquiry → offer → **contract conclusion** → **delivery/fulfilment** (packing, loading, shipping the goods) → billing/invoice → service/support.

Model	Between	Example
B2B	Business ↔ Business	Raw material portals
B2C	Business → Consumer	Amazon, Flipkart
C2C	**Consumer ↔ Consumer**	**OLX** — individuals selling to individuals
C2G / G2C	Citizen ↔ Government	Tax portals

(1) In the stakeholder letters, **'B' (business) includes the producers** — the organisations that make or supply goods.

(2) **Key drivers** of e-business: commercial benefits, requirements of customers and suppliers, and a skilled committed workforce. ("Supplementary workforce" is not a driver.)

(3) **Barriers**: the micro-environmental (external) barrier is **inadequate internet infrastructure**; culture is macro-environmental; business models and lack of experts are internal.

(4) Advantages: many payment modes, worldwide customer reach, digitalisation. A recognised **disadvantage: no universal standard for quality and reliability**.

(5) Delivery of an ordered item depends on its **size, urgency and nature — not on its cost**.

(6) **M-commerce**: mobile security is its prominent impediment; it reduces transaction cost through streamlined processes; mobile coupons and mobile tickets are its services — all three true.

Electronic Payment Systems (EPS): the types include **smart cards, e-wallets and Electronic Funds Transfer (EFT)** — all three.

Online trading procedure: open a **Demat + trading account** → **allocate funds** from the bank account → place the order, and once confirmed it goes to the **stock exchange** through the trading system — all three steps.

Internet business models: native internet models were born online — the **Library model** is one (subscription is not, in the asked classification); **transplanted real-world models** bring an offline idea online — the **free-trial model** ("try before you buy").

Note: **K. Vaitheeswaran** is called the **father of e-commerce in India** (founder of Fabmart/Indiaplaza). Website visibility grows with a guessable name and fresh attractive content; social-network icons also help.

Exam Points — 12.9

- ✦ OLX = C2C. 'B' includes producers. Packing/loading/shipping = delivery/fulfilment step.
- ✦ Micro-environmental barrier = internet infrastructure. No universal quality standard = disadvantage. Delivery does not depend on cost.
- ✦ EPS types: smart cards, e-wallet, EFT. Online trading = Demat account + fund allocation + order to exchange.
- ✦ Native internet model = Library model; transplanted real-world model = free trial. Father of Indian e-commerce = K. Vaitheeswaran.

Quick Revision — 30 points

1. Software engineering aims at software meeting user requirements within budget and schedule. SDLC: Analysis → Design → Coding → Testing → Maintenance.
2. Feasibility = technology + finance + time + resources; project size → technical feasibility. SCM identifies, controls, audits, reports changes.
3. Reverse engineering recovers design from code; re-engineering rebuilds the system in a new form.
4. Waterfall (Royce): linear, requirements frozen, low cost, long duration, high risk; fails when requirements change.
5. Prototype: quick design → build → evaluate → refine → engineer product; best for changing requirements; serves demonstration + user testing + feasibility together.
6. Incremental = Linear + Prototyping. RAD = high-speed waterfall; phases business/data/process modelling, application generation, testing; needs user participation; avoid when technical risk is high.
7. Spiral (Boehm): meta model, risk model; incremental releases; explicit risk analysis every loop; prototyping's iteration + waterfall's discipline.
8. Agile = flexibility and adaptability. 4GT = set of software tools. Generic process: Communication, Planning, Modelling, Construction, Deployment.
9. MVC = Model View Controller. Behaviour model shows response to external events.
10. Requirements engineering: feasibility → elicitation & analysis → specification → validation. Brainstorming = group elicitation.
11. SRS: black-box spec; functional + non-functional + goals; no algorithms. Non-functional sub-types: product, organisational, external.
12. DFD = bubble chart; data store = data repository; Level-0 = context diagram. UML = Unified Modeling Language; sequence diagram = objects on top.
13. CASE = Computer Aided Software Engineering — improves quality, cuts drudgery and maintenance cost. Linker resolves external symbols.
14. Cohesion = functional strength (high is good); logical cohesion = similar operations grouped. Coupling = interdependence (low is good); common coupling = shared global data.
15. Cohesion ladder: functional > sequential > communicational > procedural > temporal > logical > coincidental. Coupling ladder: data < stamp < control < common < content.
16. Design pattern = standard solution to a common problem. Partitioning: horizontal + vertical. Pipe-and-filter = chain of discrete transforms.
17. Verification = building the product right (spec); Validation = building the right product (need) — Boehm.
18. Testing levels: unit → integration → system → acceptance. Development testing = unit + component + system; release testing before acceptance, black-box, by developer.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

19. White box = glass box (internal logic). Black box from specification; methods = equivalence partitioning + BVA; BVA of 1–4000 → 0, 1, 4000, 4001.

20. Alpha = developer's site; Beta = customer's site. Regression = old features still work. Recovery testing forces failure. Mutation = fault-based. TDD = tests first. Walk-through = informal, after coding. Big-bang = all at once, not incremental.

21. SQA = standards + processes (QA prevents, QC detects). Test goal: maximise defects found, minimise test cases.

22. Quality factors: efficiency = time + resource behaviour. Process quality excludes portability. ISO 9000 = process guidelines.

23. CMM = Capability Maturity Model: Initial → Repeatable → Defined → Managed → Optimizing.

24. Patch = emergency fix (bugs/vulnerabilities); updates = patching. Maintenance: corrective, adaptive, perfective, preventive.

25. Gantt → planning phase; PERT = boxes + arrows (activities + dependencies); Gantt derivable from PERT only. W5HH = all seven questions. Risk: identify → analyse (likelihood + consequences) → plan → monitor; handle by accept/avoid/transfer.

26. COCOMO (Boehm) = Constructive Cost Model; E = a(KLOC)^b; 3.2(KLOC)^1.05 = Boehm simple model; modes organic (small, familiar), semidetached, embedded. Not a decomposition method.

27. FP measures size from functionality, pre-code, language-independent. Direct measures: LOC, cost, effort. Indirect: functionality, quality. Effort = person-months. Fog index = readability. FURPS has no portability.

28. MTTF = Mean Time to Failure; MTTR = repair; MTBF = MTTF + MTTR; ROCOF = Rate of Occurrence of Failure.

29. TPS stores transactions; MIS = planned collect-store-disseminate system (resources: hardware, software, people, data; sources internal + external); DSS aids decisions (UI + knowledge engine); OAS = office tasks. Strategic info = long-term policy. Tacit knowledge = personal, hard to formalise. Expert system = knowledge base + inference engine + UI.

30. E-commerce: OLX = C2C; B includes producers; packing/shipping = fulfilment; EPS = smart cards + e-wallet + EFT; father of Indian e-commerce = K. Vaitheeswaran; m-commerce impediment = mobile security.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

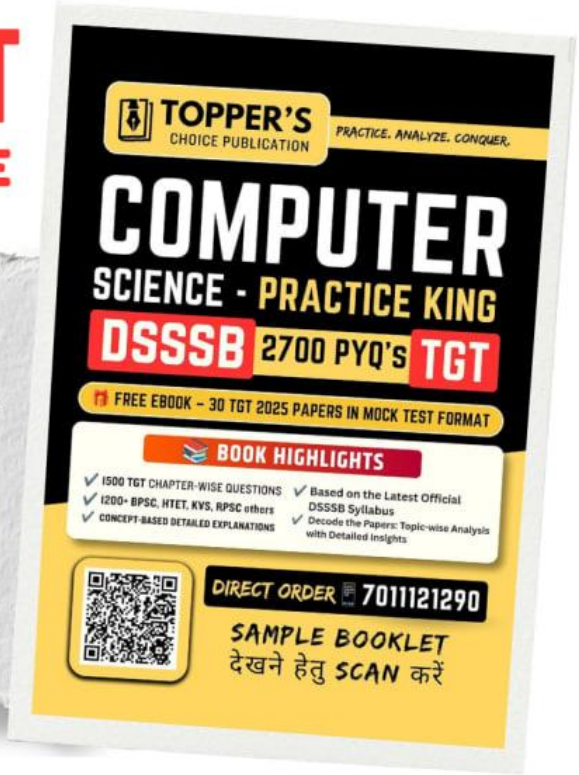
Call/Message: 7011121290 **Available @ Amazon:** https://amzn.in/d/07E04o9B

7011121290

📚 STUDY THE PATTERN, NOT THE SYLLABUS

DSSSB -TGT
COMPUTER SCIENCE

रटने वाली नहीं,
EXAM को DECODE करने वाली BOOK!
📖 हर PYQ के साथ
Concept + Trend Analysis



Scan करें और Amazon से तुरंत ऑर्डर करें

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B